Tecnológico de Monterrey
Escuela de Ingeniería y Ciencias

Campus Santa Fe

Construcción de software y toma de decisiones

TC2005B.401

Profesor: Esteban Castillo Juárez

**“4. Normalización de la base de datos del reto apoyándose en recursos de IA generativa”**

**Grupo:** 401

**Integrante:**


| | |
|---|---|
| Daniela Angulo Solís | A01028153 |
| Sofia Campos Güitron | A01277013 |
| Ariel Pulver Srihem | A01786822 |


**Fecha de Entrega:**

18 de abril del 2026

## Gladiator – Modelo Entidad-Relación y Normalización

Nuestro juego “Gladiator” registra información detallada de cada partida jugada por el jugador, incluyendo:

*Información del Jugador y Partida:*

- Datos del jugador.
- Resultados de cada partida.

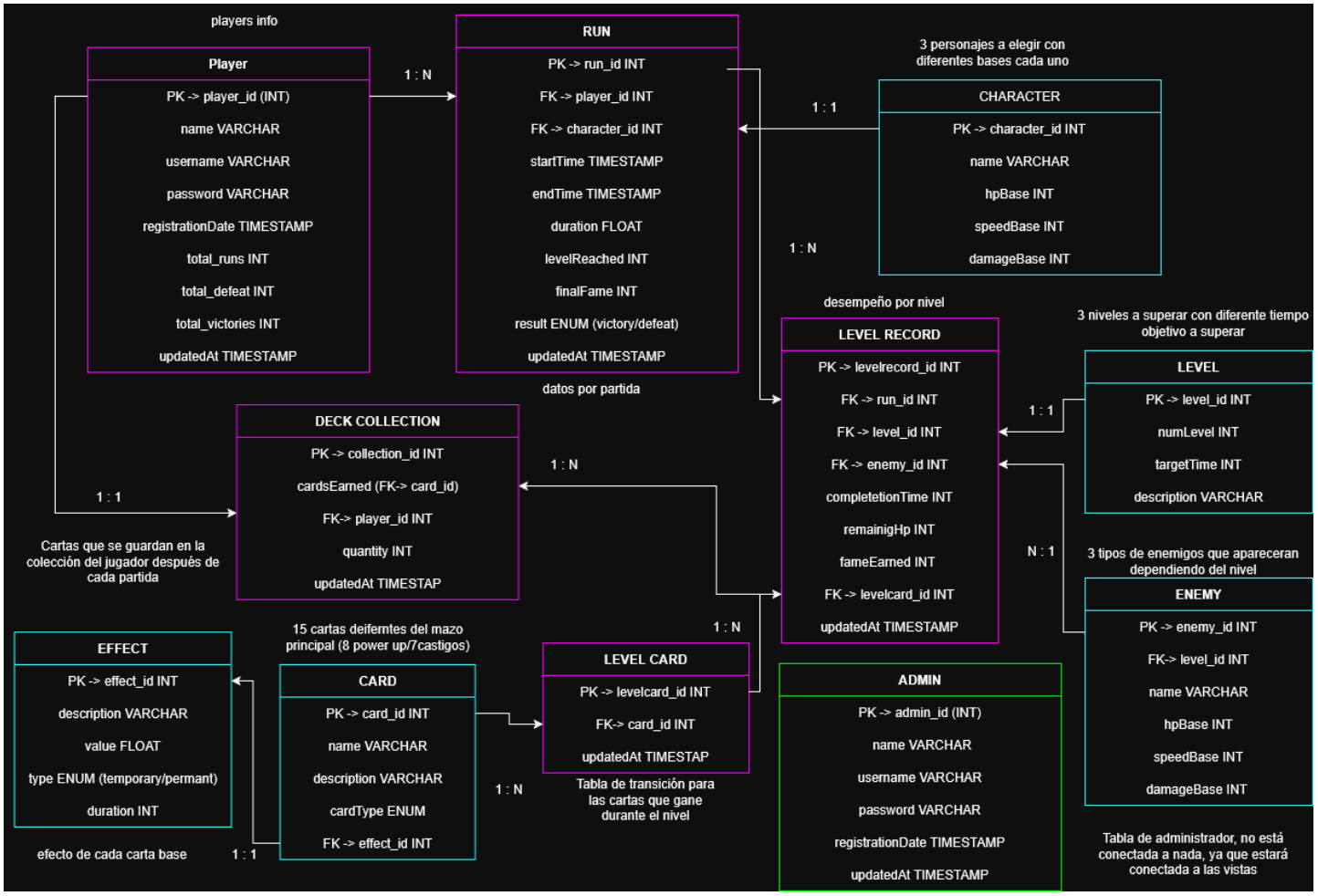
*Interacciones durante la Partida:*

- Arquetipo seleccionado al inicio.
- Progreso y avance a través de los niveles.
- Enemigos que aparecen y con los que se interactúa.
- Uso de cartas (como *power-ups* o castigos).

*Efectos:*

- Modificación de la fama del jugador.

## Modelo conceptual (Entidades y relaciones)



## *Entidades principales*

- *Player*
  Guarda la información demográfica del usuario y sus estadísticas generales (runs, victorias, derrotas).

- *Run*
  Representa cada partida que un jugador juega. Está relacionada con un jugador y un personaje.

- *Character*
  Define las características iniciales (vida, velocidad, ataque) de cada personaje (Guerrero/ Lancero/ Pesado). Se separa para que funcione como catálogo y no repetir estos valores en cada partida.

- *Level*
  Define los niveles del juego (1, 2 y 3) y sus tiempos objetivos, cada nivel tiene características extras al anterior que son especificadas en la descripción.

- *Level Record*
  Representa el desempeño de un jugador en un nivel dentro de una partida. Aquí se guarda información de otras tablas como el número de partida, de nivel y de enemigo activo, pero también información única de la tabla como tiempo de juego, fama obtenida, vida restante del jugador y las cartas que se añadieron.

- *Level Card*
  Tabla de transición para poder guardar las múltiples cartas que puede ganar un jugador durante la partida.

- *Deck Collection*
  Registra el pool de cartas que el jugador puede usar de acuerdo a su estrategia, dado que solo puede haber 6 cartas en el deck, obliga al jugador a usar sus cartas en la partida, dichas cartas se eliminaran de la colección para que nuevas puedan entrar.

- *Card*
  Define todas las cartas del juego (power-ups y castigos), que es lo que hacen y el efecto que tienen.

- *Effect*
  Representa que efecto tiene cada carta, cómo se aplica ese efecto y el valor que modifica (e.g. si sube la velocidad un 20%, en este campo iría 1.20 que es el valor por el que se multiplicará la velocidad) y por último si es temporal (tiene una duración limitada) o permanente (una duración de NULL que durará por el nivel).

- *Enemy*
  Define las características de los 3 enemigos (León, Tigre y Jabalí) y en qué nivel aparecen.

- *Admin*
  Define la información de los administradores, si la base detecta (mediante un trigger de autentificación) que la persona que inicio sesión es un admin, mostrará las estadísticas generales (vistas).

*Relaciones*

- Un Player puede tener muchas Runs
- Una Run usa un solo Character
- Una Run tiene varios Level Records
- Cada Level Record pertenece a un Level
- Un Level Record puede tener muchas Level Cards
- Muchos Level Record tienen un solo tipo de Enemy
- Una Card puede pertenecer a varios Level Card
- Una Card tiene solo un Effect
- Un Effect le pertenece a una sola Card
- Una Card puede aparecer en muchos Deck Collection
- Un Player solo puede tener un Deck Collection

Este diseño nos permite separar correctamente la información y evitar duplicarla.

## 4. Normalización

La normalización fue clave para organizar bien la base de datos de nuestro proyecto, ya que nos ayudó a evitar repetir información y a que cada tabla guardara solo lo que le corresponde, lo que hace más fácil mantener y consultar los datos sin errores. Gracias a esto, pudimos separar correctamente elementos como jugadores, partidas, niveles, cartas y enemigos, logrando una estructura más clara y ordenada. Por otro lado, el uso de herramientas de inteligencia artificial nos sirvió como apoyo para detectar posibles mejoras en el modelo, como reorganizar ciertas entidades o agregar restricciones, pero en ningún momento sustituyó nuestro trabajo, ya que cada sugerencia fue analizada y adaptada por nosotros según las necesidades del proyecto, manteniendo siempre el control sobre el diseño final.

Normalización con base en lo aprendido en clase:

Primera Forma Normal (1FN)

Se cumple porque:

- Todos los atributos son simples (no hay listas dentro de una celda)
- Cada dato se guarda en un solo lugar

Por ejemplo: en vez de guardar varias cartas en una columna, se usa la tabla CartaUsada

Segunda Forma Normal (2FN)

Se cumple porque:

- Todas las tablas tienen clave primaria
- No hay atributos que dependan solo de una parte de la clave

Por ejemplo: en Level Record, todos los datos dependen completamente de su ID

Tercera Forma Normal (3FN)

Se cumple porque:

- No hay dependencias indirectas entre atributos
- Cada tipo de información está separado en su propia tabla

Algunos ejemplos de esto:

- Arquetipo está separado para repetir stats en cada partida
- Carta está separada de su uso para evitar duplicarse
- Enemigo está separado para evitar repetir datos en cada nivel

Normalización recomendada por la IA

Con apoyo de herramientas de inteligencia artificial, se realizaron mejoras adicionales al modelo para optimizar su estructura y claridad:

- **Uso de tablas intermedias para relaciones muchos a muchos:**
  Se reforzó el uso de tablas como Level Card para modelar correctamente relaciones donde múltiples elementos interactúan (por ejemplo, múltiples cartas en un nivel).
- **Separación de catálogos:**
  Se mantuvieron entidades como Character y Enemy como catálogos independientes, ya que sus atributos no cambian por partida, evitando redundancia de datos.
- **Mejor definición de restricciones:**
  Se propuso el uso de restricciones como:
  - PRIMARY KEY para identificación única
  - FOREIGN KEY para mantener integridad referencial
  - CHECK para validar valores (por ejemplo, vida, fama, cantidades)
  - UNIQUE para evitar duplicados (como usernames)
- **Refinamiento del modelo lógico:**
  Se mejoró la estructura separando conceptos como:
  - Definición de cartas (Card) vs uso en partida (Level Card)
  - Desempeño por nivel (Level Record)
    Esto permite una mejor organización y evita redundancia.

**Justificación del diseño**

El diseño se enfoca en separar correctamente los conceptos del juego para evitar redundancias y mejorar la integridad de los datos.

*Separación de entidades*

Se decidió separar entidades como:

- Arquetipo
- Carta

- Enemigo

Porque sus datos no cambian por partida. Si se guardaran dentro de otras tablas, habría una duplicación que no es necesaria.

### *Uso de tablas intermedias*

Las tablas cartaUsada y EnemigoUsado se crearon para resolver relaciones de tipo muchos a muchos.

Por ejemplo:

- Un nivel puede tener muchas cartas
- Una carta puede aparecer en muchos niveles

Sin estas tablas, la base de datos tendría inconsistencias o duplicación de datos.

### *Integridad de datos*

Se agregaron restricciones como:

- PRIMARY KEY: identifica cada registro
- FOREIGN KEY: mantiene relaciones válidas
- CHECK: asegura valores correctos (vida, fama, etc.)
- UNIQUE: evita duplicados

Esto nos asegura que la base de datos no tenga datos inválidos

### *Evitar redundancia*

El diseño evita repetir información como:

- Estadísticas de arquetipos
- Datos de cartas
- Información de enemigos

Esto mejora el uso de memoria, la velocidad de consultas y el mantenimiento del sistema